

Java vs Kotlin — Manejo de Errores y Excepciones

Filosofía general

Java: checked + unchecked, try/catch/finally, try-with-resources.
Kotlin: no checked exceptions, promueve Result, sealed classes, runCatching.

try/catch/finally

Java:
`try { ... } catch(IOException e) { ... } finally { ... }`
Kotlin:
`try { ... } catch(e: IOException) { ... } finally { ... }`

Multi-catch

Java: `catch(IOException | SQLException e) { ... }`
Kotlin: no multi-catch, usar varios catch.

Recursos

Java: try-with-resources (AutoCloseable).
Kotlin: use `{ }` para cerrar recursos automáticamente.

Manejo funcional

Kotlin: `runCatching { risky() }, Result<T>` con `onSuccess / onFailure`.

Sealed classes

Kotlin: sealed interface `FileError { ... }` + when exhaustivo.
Java 17+: sealed interfaces y switch exhaustivo.

Buenas prácticas

Java: checked para E/S, unchecked para errores de programación, try-with-resources.
Kotlin: usar Result o sealed en dominio, use `{ }` para recursos, cuidado interoperabilidad.